

Workbook – Day 1

From Traditional SDLC to AI-Driven Software Engineering - Where should we use AI?

Thời lượng: 60 minutes

Tổng quan workshop

Sự xuất hiện của Generative AI đang thay đổi cách các hoạt động phát triển phần mềm được thực hiện. AI có thể hỗ trợ phân tích yêu cầu, đề xuất phương án thiết kế, sinh mã nguồn, tạo test case, review code và hỗ trợ nhiều hoạt động khác trong Software Development Lifecycle (SDLC). Tuy nhiên, việc đơn thuần đưa các công cụ AI vào từng hoạt động hiện có không đồng nghĩa với việc tổ chức đã chuyển đổi sang AI-Driven SDLC. Để khai thác hiệu quả AI trong phát triển phần mềm, nhóm phát triển cần xem xét lại toàn bộ quy trình: AI nên tham gia vào đâu, AI nên đảm nhận vai trò gì, con người cần duy trì trách nhiệm nào, và những điểm kiểm soát và xác thực nào cần được thiết lập.

Trong workshop này, học viên sẽ làm việc theo nhóm trên một case study xuyên suốt. Mỗi nhóm sẽ bắt đầu bằng việc phân tích quy trình phát triển phần mềm hiện tại, xác định các khó khăn và điểm nghẽn trong từng hoạt động của SDLC. Từ kết quả phân tích, nhóm sẽ khám phá các cơ hội ứng dụng AI, xác định trách nhiệm phù hợp giữa con người và AI, và đề xuất một quy trình phát triển phần mềm mới dựa trên sự cộng tác Human-AI.

Case study workshop

Một bệnh viện đa khoa đang vận hành hệ thống hỗ trợ quản lý lịch khám và các dịch vụ liên quan đến bệnh nhân (*Hospital Appointment and Patient Service Management System*). Hệ thống hiện tại cho phép bệnh nhân tìm kiếm thông tin bác sĩ và chuyên khoa, đặt lịch khám, thay đổi hoặc hủy lịch hẹn, theo dõi trạng thái lịch khám và nhận thông báo từ bệnh viện. Bác sĩ và nhân viên bệnh viện sử dụng hệ thống để quản lý lịch làm việc, theo dõi các lịch hẹn và điều phối hoạt động khám bệnh. Khi một khung giờ khám không còn khả dụng do bác sĩ thay đổi lịch làm việc hoặc bệnh nhân hủy lịch, nhân viên bệnh viện phải xử lý các thay đổi liên quan và cập nhật lịch khám cho các bệnh nhân bị ảnh hưởng. Trong những thời điểm có nhu cầu khám cao, một số khung giờ hoặc bác sĩ có thể không còn lịch trống. Bệnh nhân có nhu cầu đặt lịch được đưa vào danh sách chờ (*Waiting List*) và chờ một khung giờ phù hợp trở nên khả dụng.

Yêu cầu thay đổi

Để cải thiện khả năng quản lý lịch khám và sử dụng hiệu quả các khung giờ khám còn trống, bệnh viện quyết định phát triển một chức năng mới:

Quản lý linh hoạt việc thay đổi lịch khám và danh sách chờ

Chức năng mới này hỗ trợ việc **quản lý các thay đổi lịch khám** và **danh sách chờ**. Khi một khung giờ khám trở nên khả dụng, hệ thống cần xác định các bệnh nhân phù hợp trong danh sách chờ, và cung cấp khung giờ đó cho bệnh nhân.

Việc lựa chọn bệnh nhân cần xem xét dựa trên các thông tin liên quan như: mức độ ưu tiên, thời gian tham gia danh sách chờ và sự phù hợp giữa nhu cầu khám của bệnh nhân với khung giờ khả dụng. Bệnh nhân có thể chấp nhận hoặc từ chối khung giờ được đề xuất. Nếu bệnh nhân từ chối hoặc không phản hồi trong một khoảng thời gian nhất định, khung giờ có thể được cung cấp cho bệnh nhân khác trong danh sách chờ. Hệ thống cũng cần xử lý các tình huống thay đổi lịch khám do bệnh nhân, bác sĩ hoặc bệnh viện khởi tạo, đồng thời hạn chế các xung đột lịch và đảm bảo các bên liên quan được thông báo về những thay đổi quan trọng.

Activity 1 – Phân tích quy trình SDLC hiện tại và các điểm đứt gãy thông tin (20p)

Mục tiêu

Sau activity này, nhóm cần:

- Hiểu quy trình SDLC hiện tại khi phát triển chức năng mới.
- Xác định các artifacts được tạo ra ở từng phase.
- Phân tích những thông tin cần được chuyển giao giữa các phase.
- Xác định các điểm có nguy cơ mất thông tin (Context Drift).
- Chuẩn bị đầu vào cho Activity 2.

Bối cảnh

Sau khi nhận được yêu cầu thay đổi từ bệnh viện, nhóm phát triển bắt đầu triển khai chức năng mới. Quy trình hiện tại của công ty vẫn là quy trình SDLC truyền thống.

- Business Analyst tiếp nhận yêu cầu.
- Architect thiết kế giải pháp.
- Developer hiện thực chức năng.
- Tester xây dựng test case và kiểm thử.
- Technical Lead review trước khi triển khai.

Hiện tại mỗi thành viên đều có thể sử dụng một AI Assistant riêng để hỗ trợ công việc của mình (ví dụ ChatGPT, Claude, Copilot, Gemini...), tuy nhiên các AI này **không chia sẻ ngữ cảnh với nhau** và chỉ làm việc dựa trên thông tin mà người dùng cung cấp.

Câu hỏi đặt ra là: Trong quy trình này, những thông tin nào cần được duy trì xuyên suốt giữa các phase để AI và con người có thể cộng tác hiệu quả?

Nhiệm vụ

Bước 1: Chọn những thông tin quan trọng nhất, giả sử ở các phase Requirement Analysis, Design, Development có những thông tin sau được tạo ra:

Requirement Analysis	Design	Development
☐ Business goals	☐ Architecture Decision	☐ Source code
☐ Functional requirements	☐ API Contracts	☐ Logging
☐ Business rules	☐ Database Schema	☐ Exception Handling
☐ User stories	☐ Component Responsibilities	☐ Configuration
☐ Acceptance criteria	☐ Design Constraints	☐ Known Limitations
☐ Priority	☐ Technology Stack	☐ Technical Assumptions
☐ Business constraints	☐ Security Design	☐
☐ Stakeholders		☐

Nếu chỉ được chọn 3 thông tin quan trọng nhất, để chuyển sang phase tiếp theo, input đầu vào cho AI, bạn hãy lựa chọn cho từng bước chuyển đổi sau. Hãy giải thích cho sự lựa chọn của nhóm mình.

- 1) Requirement → Design
- 2) Design → Development
- 3) Development → Testing

Bước 2: Giả sử các tình huống sau

- 1) Developer cần thông tin nào nhất, theo nhóm điều gì có thể xảy ra với kết quả từ AI nếu không có thông tin đó?
- 2) Tester cần thông tin nào nhất, điều gì xảy ra nếu không có hoặc không đủ?
- 3) Architect cần thông tin nào nhất, điều gì xảy ra nếu không có hoặc không đủ?

Bước 3: Tổng hợp, sau khi hoàn thành hãy thống nhất 03 thông tin quan trọng nhất cần được duy trì xuyên suốt SDLC

Thứ hạng	Thông tin	Vì sao
1		
2		
3		

Bước 4: Thảo luận nhóm

- Câu hỏi 1: Trong SDLC phase nào tạo ra nhiều thông tin quan trọng nhất? Tại sao
- Câu hỏi 2: Thông tin nào dễ bị mất nhất khi chuyển giao giữa các phase?
- Câu hỏi 3: Theo nhóm nguyên nhân lớn nhất khiến các nhóm phát triển phải rework là gì
- Câu hỏi 4: Khi mỗi phase chúng ta sử dụng 1AI Assistant khác nhau, AI nào sẽ gặp khó khăn nhất? Tại sao

Deliverables: Mỗi nhóm chuẩn bị 01 slide để tóm lược các thông tin trong 4 bước nói trên. Trình bày thảo luận trong 05 phút.

Activity 2: Thiết kế mô hình Human-AI Collaboration cho SDLC (30p)

Mục tiêu

Sau activity này, học viên có thể:

- Xác định những hoạt động AI có thể hỗ trợ hiệu quả trong từng phase của SDLC
- Phân biệt những quyết định AI nên thực hiện và những quyết định con người vẫn cần chịu trách nhiệm
- Hiểu khi nào AI nên đề xuất, khi nào AI có thể tự động thực hiện và khi nào con người phải chịu trách nhiệm cuối cùng
- Hiểu khái niệm Human-in-the-loop và Human-on-the-loop trong SDLC

Bối cảnh

Sau Activity 1, những thông tin quan trọng nhất cần được duy trì xuyên suốt SDLC đã được xác định, điều này giúp cho cả Human và AI cùng luôn nắm được ngữ cảnh. Tiếp theo sẽ thiết kế quy trình SDLC với sự cộng tác Human-AI

Bước 1: AI nên tham gia ở đâu

Hoạt động	AI không nên tham gia	AI hỗ trợ	AI có thể tự động
Phân tích yêu cầu	☐	☐	☐
Sinh User Story	☐	☐	☐
Đề xuất kiến trúc	☐	☐	☐
Sinh API	☐	☐	☐
Sinh mã nguồn	☐	☐	☐
Sinh Unit Test	☐	☐	☐
Code Review	☐	☐	☐
Security Review	☐	☐	☐
Sinh tài liệu	☐	☐	☐
Release lên production	☐	☐	☐

- 1) Hoạt động nào AI mang lại nhiều giá trị nhất?
- 2) Hoạt động nào AI có nhiều rủi ro nhất?

Bước 2: Phân chia trách nhiệm giữa AI và Human

Với mỗi phase của SDLC, hãy xác định:

- AI chịu trách nhiệm gì?
- Human chịu trách nhiệm gì?
- Vì sao

Human-AI Responsibility Matrix

Phase	AI responsibility	Human responsibility	Vì sao?
Requirement Analysis			
Design			
Development			
Testing			
Code Review			
Deployment			

Instructions: Hãy mô tả AI có thể làm gì?

Bước 3: Quyết định nào AI không nên thay thế con người

Đối với các hoạt động dưới đây, hãy thảo luận và đánh giá:

- AI có thể quyết định: AI có thể tự động thực hiện mà không cần con người phê duyệt
- AI chỉ đề xuất: AI đưa ra gợi ý, con người xem xét và quyết định
- Human quyết định: quyết định bắt buộc phải do con người phụ trách

Lưu ý: Không có đáp án đúng duy nhất. Nhóm lựa chọn trên mức độ rủi ro, trách nhiệm và tác động của từng quyết định.

Requirement Engineering

Quyết định	AI có thể quyết định	AI chỉ đề xuất	Human quyết định
Làm rõ yêu cầu	☐	☐	☐
Sinh user stories	☐	☐	☐
Sinh acceptance criteria	☐	☐	☐
Ưu tiên trong Backlog	☐	☐	☐
Thay đổi business rules	☐	☐	☐
Chấp nhận requirement	☐	☐	☐

System Design

Quyết định	AI có thể quyết định	AI chỉ đề xuất	Human quyết định
Đề xuất kiến trúc	☐	☐	☐
Chọn công nghệ	☐	☐	☐
Thiết kế API	☐	☐	☐
Thiết kế Database	☐	☐	☐
Thiết kế Security	☐	☐	☐
Phê duyệt Architecture	☐	☐	☐

Development

Quyết định	AI có thể quyết định	AI chỉ đề xuất	Human quyết định
Sinh mã nguồn	☐	☐	☐
Refactoring	☐	☐	☐
Sinh tài liệu kĩ thuật	☐	☐	☐
Chỉnh sửa code hiện có	☐	☐	☐
Merge Pull request	☐	☐	☐
Commit trực tiếp vào main branch	☐	☐	☐

Testing & Quality Assurance

Quyết định	AI có thể quyết định	AI chỉ đề xuất	Human quyết định
Sinh test cases	☐	☐	☐
Sinh test data	☐	☐	☐
Phát hiện bug	☐	☐	☐
Đóng bug	☐	☐	☐
Đánh giá Test Coverage	☐	☐	☐
Quyết định approve phiên bản	☐	☐	☐

Code Review & Security

Quyết định	AI có thể quyết định	AI chỉ đề xuất	Human quyết định
Phát hiện code smell	☐	☐	☐
Đề xuất Refactoring	☐	☐	☐
Phát hiện Security Vulnerability	☐	☐	☐
Chấp nhận Security Risk	☐	☐	☐
Phê duyệt Code Review	☐	☐	☐

Deployment & Operations

Quyết định	AI có thể quyết định	AI chỉ đề xuất	Human quyết định
Sinh Release Note	☐	☐	☐

Đề xuất thời điểm Release	☐	☐	☐
Triển khai Production	☐	☐	☐
Rollback phiên bản	☐	☐	☐
Xử lý Incident	☐	☐	☐
Phê duyệt Hotfix	☐	☐	☐

Câu hỏi thảo luận:

Q1: Quan sát toàn bộ SDLC, nhóm nhận thấy:

Phase nào AI có thể tự động hoá nhiều nhất

Phase nào cần có sự tham gia của con người nhiều nhất? Vì sao?

Q2: Trong các quyết định trên, nhóm hãy ranking 3 quyết định quan trọng nhất luôn phải do con người chịu trách nhiệm

1. _____
2. _____
3. _____

Giải thích.

Bước 4: Phân biệt HITL và HOTL

4.1. Xác định mô hình phù hợp với các tình huống dưới đây:

Tình huống	Human in the loop	Human on the loop
AI sinh user story, BA kiểm tra trước khi sử dụng	☐	☐
AI sinh Acceptance criteria, BA phê duyệt	☐	☐
AI đề xuất kiến trúc, Architect phê duyệt	☐	☐
AI sinh mã nguồn, Developer review trước khi merge	☐	☐
AI review Pull request, Developer chỉ kiểm tra các cảnh báo	☐	☐
AI sinh unit test, developer chỉ xem khi test thất bại	☐	☐
AI tự động sinh tài liệu kỹ thuật	☐	☐
AI tự động deploy Production	☐	☐
AI tự động rollback khi phát hiện lỗi	☐	☐

4.2. Điều gì quyết định mức độ tự động hoá?

Giả sử trong tương lai AI ngày càng chính xác hơn, theo nhóm, điều kiện nào cần được đáp ứng trước khi một hoạt động có thể chuyển từ Human-in-the-loop sang Human-on-the-loop?

Hãy đánh dấu những tiêu chí quan trọng nhất.

Tiêu chí	Chọn
AI có độ chính xác cao và ổn định	☐
AI có confidence score rõ ràng	☐
AI có khả năng giải thích kết quả	☐
Có cơ chế Audit Log	☐
Có khả năng Rollback khi xảy ra lỗi	☐
Có dữ liệu lịch sử đủ lớn	☐
Cơ chế Human Feedback liên tục	☐
Có quy trình kiểm soát chất lượng	☐
Quyết định có mức độ rủi ro thấp	☐
Quyết định dễ khôi phục nếu sai	☐

3 tiêu chí quan trọng nhất là gì?

1.

2.

3.

Deliverables: Mỗi nhóm chuẩn bị 01 slide gồm các nội dung sau

1. Human-AI Responsibility Matrix
2. 3 quyết định quan trọng luôn phải do Human quyết định
3. 3 hoạt động luôn cần Human-in-the-loop
4. 3 hoạt động cần chuyển sang Human-on-the-loop
5. AI Governance Principles – Hoàn thành các phát biểu sau

AI chỉ được phép tự động quyết định khi

Các quyết định liên quan đến nghiệp vụ phải

Các quyết định liên quan đến kiến trúc, chất lượng và an toàn phải

Mọi quyết định do AI thực hiện cần

Trong trường hợp AI và con người có kết quả khác nhau